

박지원 포트폴리오

Unity / C# Client Programmer

2026

목차 (지금은 계획서 느낌으로)

이력서

나는 왜 전 직장에서 버티지 못했을까? 열심히는 했지만.

- 크게 보면 아래 5가지
- 과거의 내 모습을 완벽하지 않지만 바꾸려고 노력한 그걸 기반으로 적어보았다.
- 그리고 지금은 해결한 것 같다.
- 개발자가 아닌 한 명의 인간으로 달라진 마인드를 꼭 5번에 잘 적어보자.

1

2

3

4

5

마치며

- 저는 한 번 가능성을 인정받았지만,
- 그 가능성을 실무 실력으로 충분히 전환하지 못했던 경험이 있습니다.
- 게임, 개발은 서로 다른 영역처럼 보였지만, 잘하게 되는 방식은 비슷했다.

반복 실행만으로는 부족했고, 결과를 보고 원인을 분석하고, 나만의 기준으로 압축한 뒤, 다음 실행에 적용해야 했다.



이력서

송실대학교 컴퓨터학부

2016.03 ~ 2022.02 : 컴퓨터학부 학사

2017.04 ~ 2019.01 : 군 복무



풀어비스 인턴

플랫폼 프로그래밍 2팀

2021.12 ~ 2022.02



넥슨게임즈 넥토리얼 2기 및 정규직

제우스 프로젝트_Unity 클라이언트 프로그래머

2022.11 ~ 2024.04



[Project ZEUS => GODSOME]

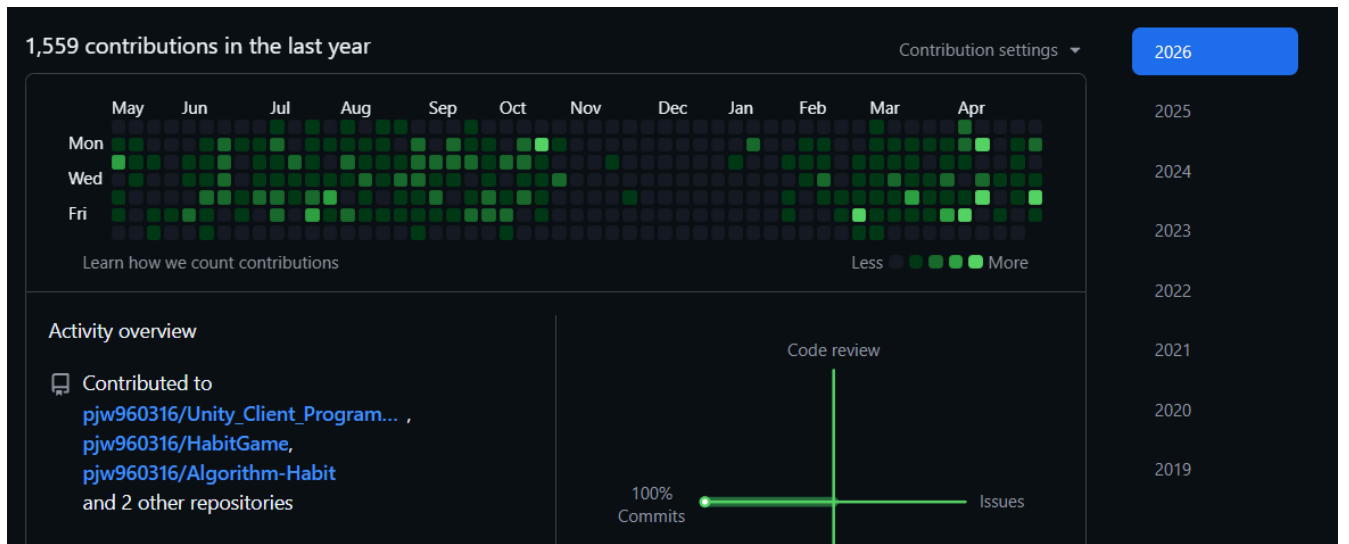
(https://www.nexongames.co.kr/en/game/project_zeus_en.php)

X1

2024.05 ~ 2024.12

X2

2025.01 ~ 2026.현재



2025년 11월 ~ 2026년 1월은 이력서 첫 제출, 포트폴리오 정리, 필기 시험 준비를 병행하던 시기입니다.

1 게임 개발 프로젝트 설명 -> 코드가 가장 많이 들어갈

개발 의도

설계 방향

- mvp pattern
- Design Pattern
- 기존의 문제점을 해결하기 위함.
- 주니어 개발자의 경우 특히 그렇다. 경험이 적은 상태에서 AI가 생성한 코드를 리뷰하는 건, 아직 운전을 배우는 중인 사람에게 자율주행차의 판단을 평가하라고 하는 것과 비슷한 면이 있다. 직접 핸들을 잡아봐야 도로 위의 감각이 생기는 것처럼 직접 코드를 짜봐야 구조에 대한 감각이 생긴다.

구현을 하며 느낀 문제점

- c# 구현력이 약함.

2 설계하며 고민하며 구현한 과정

단순 기능 구현 연습은 의미가 없다.

- 학교나 회사에서 디자인패턴 책은 항상 구매했었다. 항상 쉬운 패턴은 열심히 읽고 어려운 패턴은 외웠었습니다.
- 파트장님께서 디자인패턴은 이론적으로 공부하는 게 아니라, 구현하다보면 어느덧 사용하고 있는 자신을 바라보는 거라고 했다.
 - 당시에는 이해가 되지 않았으나 지금은 조금 이해가 됩니다.
- 롤토체를 하면서 배운 게 있습니다. 무수히 많은 상황 같아도 결국은 정해진 몇 가지 패턴이란 것을. 그리고 그 상황마다의 패턴을 기록해 놓는 게 핵심인 것 같아요.
 - 이 개념이 디자인패턴과 크게 다르지 않다고 생각합니다. 유니티 개발에서 매 번 막히는, 고민하는 설계도 사실 지금 제 수준에서는 몇 가지 였고요.
 - 그걸 고민하다보니 자연스레 고수분들은 이럴 때 어떻게 할까? 아 이게 디자인패턴이구나.

문제 해결을 위해 기록한 내용

- 부족하지만 내부 구조를 구현하려고 삽질을 좀 많이 했습니다. 그러다 보니 구현을 많이 못하고 설계만 하기도 했죠.
- 하지만 그 과정에서 클래스 구조의 확장성 문제를 직접 마주했고, 의존성 방향, 자료구조, 상속 구조, 다형성 적용 방식을 바꿔가며 여러 설계 시도를 했습니다.
- 예전에는 개발과정 없이 그냥 책만 읽었으니 도움이 안 됐습니다. 그러나 제가 삽질하며 도달한 구조를 책에서 찾아보니 특정 디자인 패턴과 닮아 있다는 것을 알게 되었고, 제가 삽질한 과정과 비교를 해봤습니다.
- 그러다 알게 된 건 디자인 패턴도 정답이 아니고 대단한 기법도 아니다. 결국 모두 trade-off고 장단이 있다는 것 같아요.
- 분기를 줄이고 싶어서 전략패턴을 만들었지만 결국 코드가 복잡해지고, 옴저버 패턴을 만드니 의존성을 줄지만 디버깅이 어렵고. 하지만 분명 계속 개발하면서 이걸 신경쓰면 저만의 패턴이 생기는 거죠.

⚠ 문제 상황

- 예전에는 시니어 분들의 코드를 암기했던 거 같음
- 그리고 어차피 신입에게 주어지는 UI 쪽 View 코드는 구현하기 쉬웠음.
- 팀장님은 그러면서도 내가 내부 코드를 읽길 바랬음. 하지만 나는 그러지 못했음.
- 회사에서 디자인 패턴 책만 3번 샀었던 거 같음. 매 번 눈으로만 읽으니까 뭐 어찌라고? 싶었음. 그러다가 파트장님이 디자인패턴은 공부하는 게 아니라 짜다보면 필요함을 느끼고 내가 쓰던 게 그런 거구나 알게되는 거. 당시에 는 전혀 이해가 안 갔음.

✅ 해결 과정

- 회사를 돌아켜보면 어쨌든 구현시에 마주하는 문제는 크게 몇 가지 상황이었다. 롤체로 따지면 덱을 몇 개 정해놓고 그에 맞는 최선의 플레이 방법이 있는 거 처럼. 근데 나는 정해 놓은 덱이 없던 것 이다.
- 아 그러면 이런 상황에는 이렇게 해야 한다.가 없구나. 코테에서도 그랬다. 나름의 원칙과 기준이 있어야 함.
- 유니티에서 처음에 생각한 게 mvp ->
- 그러다가 나름의 룰. 아 근데 좀 고수들의 룰을 알고 싶은데 -> 아 그게 디자인 패턴이구나를 드디어 이해했다.
- 그리고 일단 구현을 했다. strategy pattern -> 이걸 책을 보고 한 게 아니라. 그냥 뭐 분리를 하고 싶어서 2일 정도를 설계만 했었다. 그치만 진전이 나가지 않았고, 아 분명 이런 상황은 많을 텐데. 그래서 일단 내가 할 수 있는

최선으로 구현을 해 봤는데 그게 strategy pattern이랑 비슷한 거다. 그리고 예전에 사 놓은 책을 읽으니 너무 이해가 잘 되는 것 이다. 그 때 깨달았다. 이거다.

- 맨날 수학의 집합처럼 싱글턴만 보던 내가 드디어 다른 패턴을 이해하기 시작

문제점

- 근데 과설계 병에 걸렸다. -> 그리고 이것도 해결했다.
 - 생각해보면 매 번 회사에서는 내게 이런 고민을 할 시간을 주지 못한다.

3 AI 시대의 구현력.

⚠ 문제 상황

- 한 달에 한 번씩 회사에서 책을 사줬다. 매번

✅ 해결 과정

1. C# 제프리 문서 학습
2. 글로 읽지 않고 코드를 짜기 시작
3. AI 시대에 구현력이 중요한가에 대한 의심

🎮 게임 개발 적용

🚀 깃허브 링크



메모만 하는 공부가 아닌 나만의 글로 요약한 기록으로

⚠ 문제 상황 : 무언가를 많이 적어 메모는 많지만

1. 메모만 하고 기록을 하는 과정이 없었다.

- 인턴 중간 평가 때 팀장님께 혼난 기억이 있다
 - 너는 뭘 공부했니? 나도 뭔가를 많이했었다.
 - 저는 vscode에 열심히 기록하고 있어요 (사실 메모 수준)
 - 보여줄래?
 - 어... 그니까... 적기만 하고 본 적이 없으니... 모바일로 아이디 로그인도 못했던. 실제로 뭔가 한 것도 별로 없었고 제대로 정리한 거도 없고.
- 우리 코드 UI 어떻게 구성되었니? 어 나는 UI 쪽 코드 계속 봤고 메모도 했는데? 조금만 깊게 물어보시면 제대로 아는 게 하나도 없더라.
- 2023년에도 회사에서 gpt 유료로 지원해줬었다. 그래서 당시부터 나는 ai를 적극적으로 사용했다.
 - ai가 답을 환상적으로 만들어주니 나의 메모는 대부분 gpt 답변 캡처였던 거 같다. 그게 내 지식이라고 착각했고
 - 그러니

2. 많이 적혀있지만 그게 어디 있는 지 나도 모른다.

- 과거 문서 캡처 및 링크
- ◦ 메모는 많이 한다. 그리고 어디에도 잘 저장한다. 그러나 돌아켜보면 다시 본 기억이 없다. 그러니 실력이 늘지 않았다. 링크나 캡처는 또 엄청 한다. 가공하기 귀찮으니까
- 1. 퇴사 직전 시점에 입사 이전보다 개발자로의 경험이 늘었는가? 실력이 늘었는가? 실력에 대해서 늘었다고 생각을 못하고 무너졌던 거 같다.

✅ 해결 과정 : 나의 한 줄을 기록하고 다시 보고 고치고 고치고 고친다.

1. 현실적인 직장인의 한계에 맞게 만들어 낸 기록 시스템

- 회사에서 업무를 하면서 동시에 공부를 하기란 어렵다. 즉, 메모 -> 기록이 현실적으로 어렵다.
- 하지만 패턴화 하면 된다.

오늘의 메모

- 개발 메모

- 질문

- ✓ 매니저가 controller에게 조회하기 위한 값을 controller가 필드로 캐싱해도 되는가?

- 메모

- GPT 대화 및 공부한 내용의 1~2줄 최종 요약만 적는다.
 - 근거가 확실한 핵심 내용 & 키워드
 - 링크
 - ✓ InputController는 MonoBehaviour라서 아무 데서나 new 하기 어렵다.
InputController는 Manager에 등록된다.
외부 흐름은 Manager를 통해 Controller를 조작한다.
따라서 Controller의 public 메서드가 있어도 호출 경로를 제한하기 쉽다.
 - ✓ Controller는 Unity 세계와 가까운 계산을 합니다.
 - ✓ 크니까 controller와 manager가 모두 mousepointerpos를 들고 있는 건 괜찮다. 근데 반드
사 manager를 통해 조회가 되고 manager는 그 조회를 위해 controller의 값을 가져온다. 그
러면 콜백도 필요없다
 - ✓ 컴파일 타임에 관계가 고정되어 있다
→ 개발자가 직접 적는다
런타임에 값/상태/데이터에 따라 대상이 바뀐다
→ 매핑, Dictionary, Factory, Registry 등을 고려한다

○

- 개발 메모

- 질문

- ✓ `foreach (var pair in _dict.OrderBy(kv => kv.Key)) -> 캐싱 해?`

- 메모

- GPT 대화 및 공부한 내용의 1~2줄 최종 요약만 적는다.
 - 근거가 확실한 핵심 내용 & 키워드
 - 링크
 - ✓ compile 타임에 개발자가 타입을 알면 Generic
 - ✓ 필드 + 메서드에 공용으로 쓰는 generic은 class에 선언 메서드에서만 사용되는
generic은 method에만 선언!
 - ✓ 좋은 추상화는 누구나 직관적으로 알 수 있어야 한다.
 - ✓ 지금 딱 알았다. 확실한 거로 정리 Controller는 일단 자신을 GameManager에
게 전달하니 문제 X 문제는 연결할 Manager 객체임 싱글턴. 크니까 new도 안 먹
히고, 근데 나는 컴파일 타임에 그 Manager의 타입을 안다. 어어어어 타입???? 리
플렉션??? 이 있는데 생각해보니까 GameManager에서 Manager들의 타입을
알면 인스턴스를 리턴하는 메서드 만들면 되잖아. 그러면 두 인스턴스 알고 호출
도 되고! 나 이제 이해한 듯

○

- 우선 이렇게 노션에 적는다. gpt를 통해 공부하거나 코드를 읽으며 모르는 개념을 구글링하다보면
핵심 키워드가 생기고 그걸 적는다.

■

2. 구현과의 병행

문제점

- 이력서를 제출할 때 개발을 멈추기도 했었음.

5 깃허브 저장소 -> 레포 안내로 요약. (간단해야 한다.)

C#

이 저장소가 필요했던 이유

- 앞선 섹션에서는 실제 Unity 프로젝트와 C# 코드 중심으로 구현 역량을 설명했습니다.
- 개발자의 가장 기초
- 제프리 리처는 팀장님께서 이전에 사내 스터디에 사용하신 책이었다.
- 코딩테스트를 풀면서 실제로 배운 C# 내용도 정리

문제 해결을 위해 기록한 내용

Unity

이 저장소가 필요했던 이유

-
- 앞선 섹션에서는 실제 Unity 프로젝트와 C# 코드 중심으로 구현 역량을 설명했습니다.

문제 해결을 위해 기록한 내용

- 유니티의 API를 모두 암기할 수는 없다. 그리고 그걸 기록하는 것도 결국 암기의 영역이라고 판단한다. 물론 그게 아예 중요하지 않음은 아니다.
- 근데 경험상 유니티 개발을 하다보면 막히는 부분은 대부분 동일했고 보통 구현 과정에서 설계였다.
- 유니티 API 보다는 설계하면서 배운 내용을 기록
- MVP Pattern을 특히 강조

5 깃허브 저장소

Design Pattern

Algorithm Habit

이 저장소가 필요했던 이유

- 코테에 대한 생각
- 코테 저장 장소

문제 해결을 위해 기록한 내용

Developer Insight Journal

이 저장소가 필요했던 이유

- AI 시대에 이직을 준비하면서 현실적으로 불안을 해소 시켜 준 내용들
- 직장을 다닐 때는 시니어들과 함께 있었으나, 지금은 그러지 못함. 최소한 그걸

문제 해결을 위해 기록한 내용

- 필기테스트 보고 온 후기 -> 아 뭔가 더 같고 닮으면 100점 맞을 수 있겠는데? 아쉽지만 30점 맞던거 90점 된거 같다.
- 지금이야 내가 온전히 시간을 관리하니 가능할지 모른다. 과연 회사를 다니면서도 이 프로세스가 정상으로 동작 할까? 과거처럼 무너지지 않을까? 하지만 이젠 확신할 수 있다. 적어도 그 상황에 맞는 최선의 해결책을 찾는 능력은 생긴 것 같다. 또 수정하고 또 고치면 된다. 하다보면 바뀐다.



마치며

- 과거에는
- 역산을 하는 개발자. 문제가 터지면 왜 이 문제가 발생했을까 부터 고민하고, 현재 상황에 맞는 선택하기
- 그래서 이런 게임을 만들기로 했다.
- 그리고 이직과 그런 루프속에서 벗어나 그냥 평생 독립적으로 해보기로 했다. 회사 다녀도 꾸준히 될지는 모르겠으나. (일단)
- 체력 좋아진 거도 적자. 회사 외 적인 게 너무 중요함을 깨달음. 잠. 운동. 인간관계